

School of Computer Science

Programming for Beginners

COMP 2067

Slides prepared by Dr. Arunita Jaekel

Modified by Dr. Olena Syrotkina

Summer 2025



University of Windsor

Copyright Notice

This content is protected and may not be shared, uploaded, or distributed. Any unauthorized distribution, reproduction or sharing of this material, including uploading to **CourseHero**, **Chegg**, **StuDocu** or other websites, is **strictly prohibited** and may be subject to legal action. Students are granted access to this material solely for their personal use and may not use it for any other purpose without the express written consent of the instructor. Thank you for respecting our intellectual property rights.



What is Programming

- Topics
 - Programming Languages
 - Problem Solving Using Computers
 - Computer programs
 - Why Python?



What is a Language?

- Method of communication
- Natural Languages:
 - For human communication: spoken, written, signed ...
 - Uses **words** in a structured and conventional way
 - Examples: English, French, Spanish ...
- Programming Languages:
 - For communication with computers
 - Uses **tokens** in a structured and conventional way
 - Examples: C, C++, Java, Python ...



Natural vs Programming Languages

- Natural Languages
 - Evolved naturally
 - Building blocks: words
 - More ambiguity and redundancy
 - Looser syntax
- Programming Languages
 - Designed to express computations
 - Building blocks: **tokens**
 - Designed to be unambiguous and concise
 - Strict **syntax**



Tokens and Syntax

- English:
 - Valid tokens: red, house, is ...
 - Invalid tokens: hfksfhk, ouwoer&* ...
- Python
 - Valid tokens: print, (, +, [, if, ...
 - Invalid tokens: =\$=, !!!!, ...
- **Syntax**: pertains to tokens and their structure, i.e. how tokens are arranged. Think grammar rules.
- Examples:
 - This is a book.
 - Peace orange furiously.
 - I *is* teaching.
- Python examples:
 - $x = y + 5$ (valid)
 - $x - y = 5$ (invalid)



Computer Programs

- Computer Program:
 - a collection of *instructions* that performs a specific task when executed by a **computer** [1]
 - A detailed plan or procedure for solving a problem with a computer [2]
 - an unambiguous, ordered sequence of computational *instructions*
 - a set of *instructions* that makes a computer do a particular thing [3]
- Instruction:
 - An order given to a computer processor by a computer program [4] .
 - A single step in solving a more complex problem.
 - Analogous to a single step in a recipe, e.g. Boil two cups of water.



High and Low Level Languages

- High-level languages (HLL):
 - Most programs are written in high-level languages e.g. Python, C, Java, PHP ...
 - Easier to write (and debug) programs
 - Shorter code, less time to write
 - Portable: can run on different types of computers with little or no modifications
- Low-level languages:
 - Also called machine language or assembly language
 - Executed by computers
 - Depends on the underlying hardware, i.e. **NOT** portable.
 - Programs written in a HLL must be translated to a low-level language before they can be executed.
 - Done using **compiler** or **interpreter**

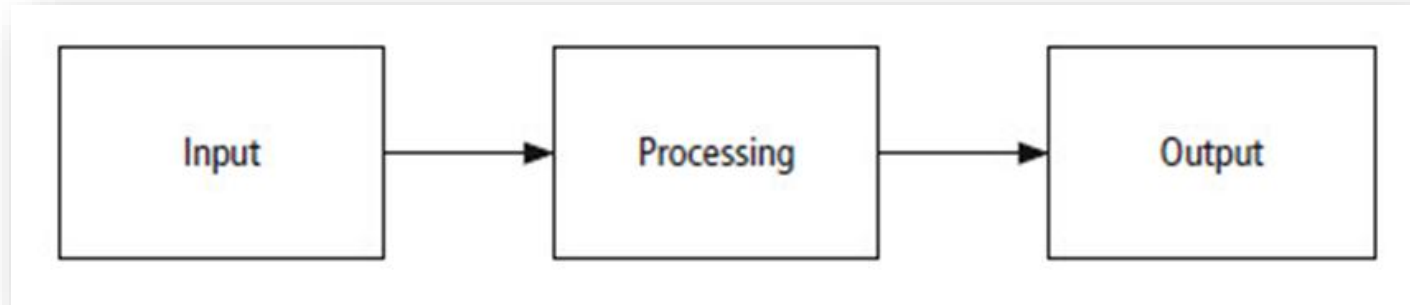


Writing Programs to Solve a Problem

1. Analyze:
 - Clearly define the problem.
2. Design:
 - Plan the solution to the problem.
 - Create an **algorithm**:
 - Break down the task into a sequence of simpler steps
3. Code:
 - Translate the algorithm into a set of instructions in a programming language.
4. Test and correct:
 - Locate and remove any errors in the program.
5. Complete the documentation:
 - Organize all the material that describes the program.
 - Include additional **comments**, as needed.



Program Components



- Input: Obtain data e.g. from keyboard, file, mouse, other devices.
- Output: Display data to user e.g. on screen, print to file, audio etc.
- Processing:
 - Store/retrieve data
 - Manipulate data e.g. perform mathematical operations
 - Conditional execution: check for certain conditions and execute appropriate set of instructions depending on the condition(s)
 - Repetition: Perform certain actions repeatedly (possibly with some variations) for a fixed number of times or until some condition is satisfied.

Why Python?

- **Easy-to-use language:** Powerful, easy to download, write and read; ideal for prototype development.
- **Large standard library:** supports many common programming tasks e.g. connecting to web servers, regular expressions, file I/O.
- **Runs on different computers and operating systems:** Windows, MacOS, many brands of Unix, OS/2, ...
- **Free software:** Free to download or use Python - the language is copyrighted it's available under **an open source license**.



How to download and install Python?

1. Go to the official Python website: <https://www.python.org/downloads/>
2. Click on the "Download" button located in the center of the screen or scroll down to the "Python Releases for Windows" section.
3. Choose the version of Python you want to download. If you're unsure, it's recommended to download the latest stable version.
4. Scroll down to the "Files" section and select the installer that's appropriate for your operating system (Windows, macOS, Linux).
5. Once the installer has finished downloading, open it.
6. Follow the prompts in the installer to complete the installation process.
7. After installation is completed, open the command prompt or terminal and type "python" to check if it was installed correctly. If Python is installed, the version number should be displayed.

Congratulations! You have successfully downloaded and installed Python on your computer.



Writing your first program on Python - I

Writing your first program on Python is a simple and exciting process. To write your first Python program, you will need to follow the steps below:

- Step 1: Install Python Before you can write your first program on Python, you need to install Python on your computer. You can download the latest version of Python from the official website: <https://www.python.org/downloads/>
- Step 2: Open the IDLE editor **after** installing Python on your computer, you can open the IDLE editor by clicking on the IDLE icon on your desktop. The IDLE editor is an interactive Python environment where you can write, test, and execute Python code.
- Step 3: Write your code In the IDLE editor, you can start writing your code. For example, you can write a simple program that prints the words "Hello, World!" on the screen by typing the following code:

```
1 print("Hello, World!")
```

Congratulations! You have successfully written and executed your first Python program. Would you like to write a more complicated program?



Writing your first program on Python - II

Here's an example program that asks for the user's name and greets them personally.

- Step 1: Open the IDLE editor.
- Step 2: Go to File -> New File (Ctrl + N).
- Step 3: Type the following code and save your file.

```
1 name = input("What is your name? ")
2 print("Hello, " + name + "! Nice to meet you.")
```

- Step 4: Go to Run -> Run Module (F5).

When you run this program, it will prompt the user to enter their name. Whatever the user types will be stored in the name variable. Then, the program will use string concatenation to combine the user's name with the greeting message and print it out.

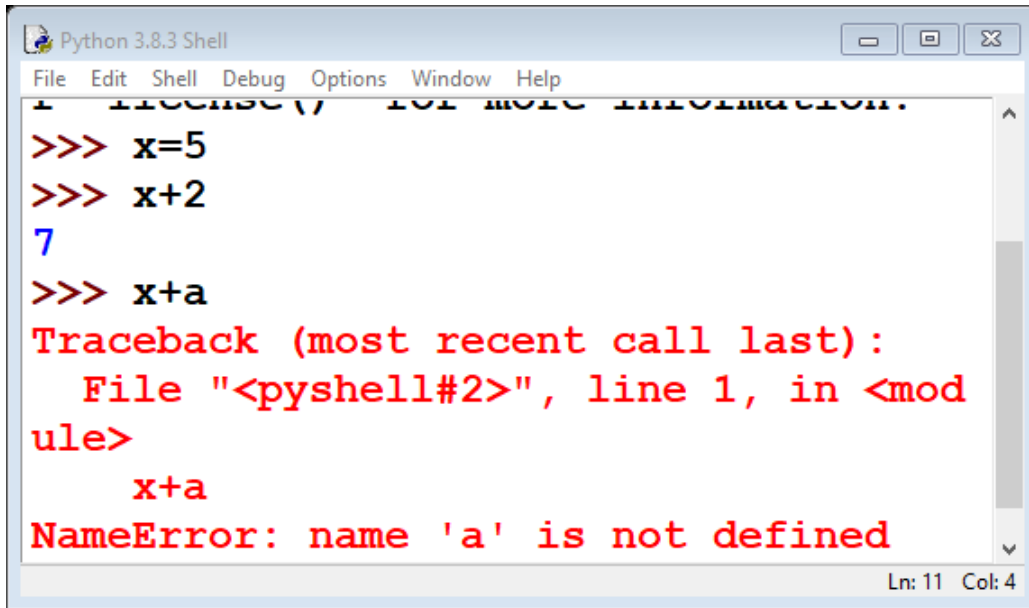
Commands used:

1. print(): This command is used to display text or values on the screen. For example, print("Hello, world!") will display "Hello, world!" on the screen.
2. input(): This command is used to accept user input. For example, name = input("What is your name? ") will ask the user for their name and store it in the variable name.

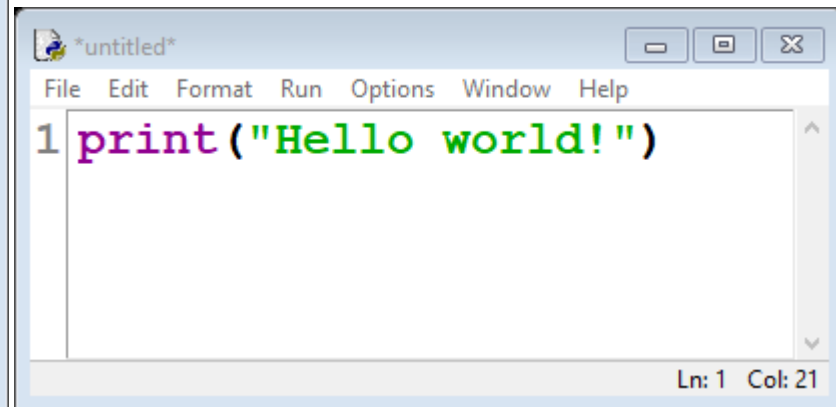


Using IDLE

- IDLE: Python's Integrated **D**evelopment and **L**earning **E**nvironment
 - Python shell window: interactive programming environment
 - colorizing of code input, output, and error messages
 - Editor window: this is where you will write your programs



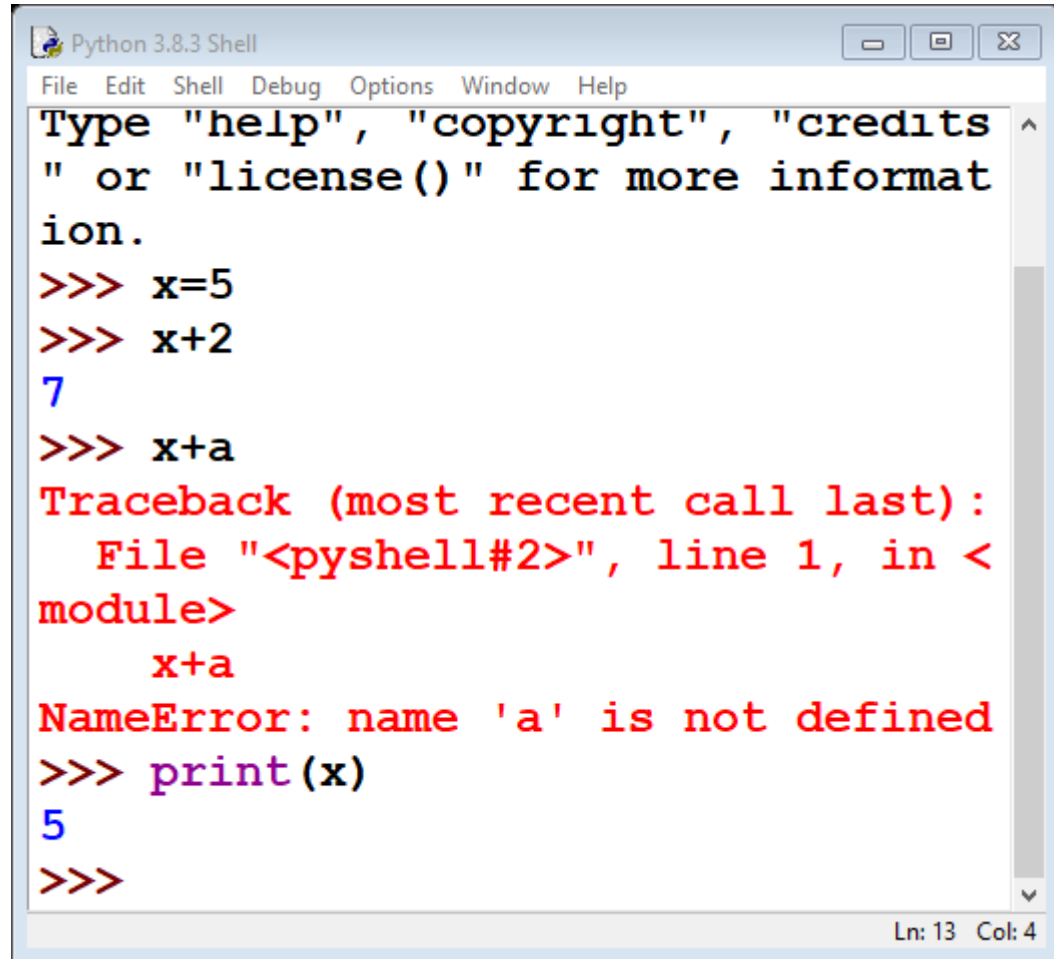
```
Python 3.8.3 Shell
File Edit Shell Debug Options Window Help
1 LICENSE (/) FOR MORE INFORMATION.
>>> x=5
>>> x+2
7
>>> x+a
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <mod
ule>
    x+a
NameError: name 'a' is not defined
Ln: 11 Col: 4
```



```
*untitled*
File Edit Format Run Options Window Help
1 print("Hello world!")
Ln: 1 Col: 21
```

Python shell window

- Can be used to directly execute simple instructions
- Convenient for testing short bits of code
- Do not use for anything longer than a few lines.

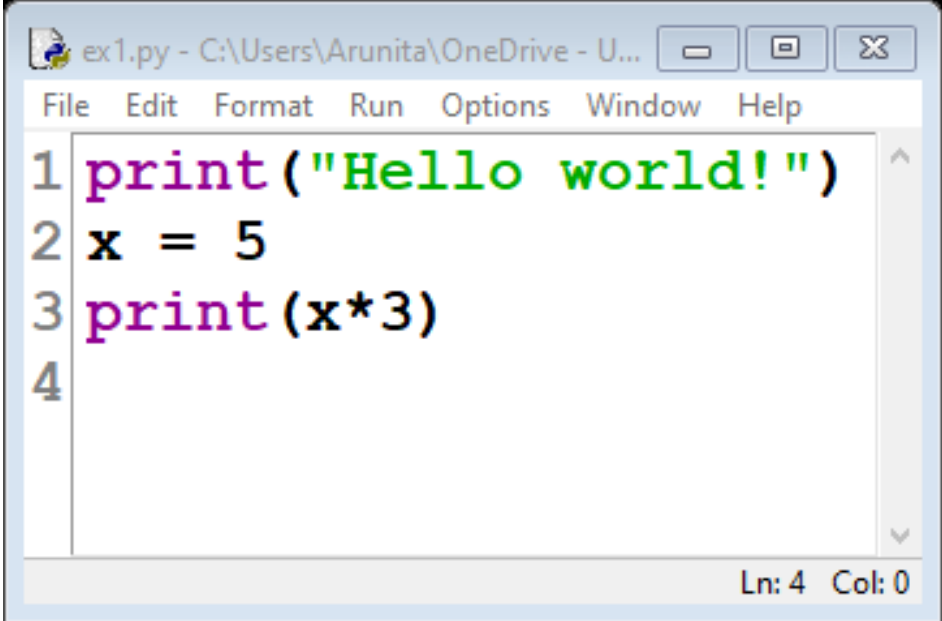
A screenshot of a Python 3.8.3 Shell window. The window has a title bar with the text "Python 3.8.3 Shell" and standard window controls (minimize, maximize, close). Below the title bar is a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The main text area contains the following text:

```
Type "help", "copyright", "credits"
or "license()" for more informat
ion.
>>> x=5
>>> x+2
7
>>> x+a
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <
module>
    x+a
NameError: name 'a' is not defined
>>> print(x)
5
>>>
```

The text is color-coded: prompts are blue, user input is black, and error messages are red. A vertical scrollbar is on the right side of the text area. At the bottom right of the window, the status bar shows "Ln: 13 Col: 4".

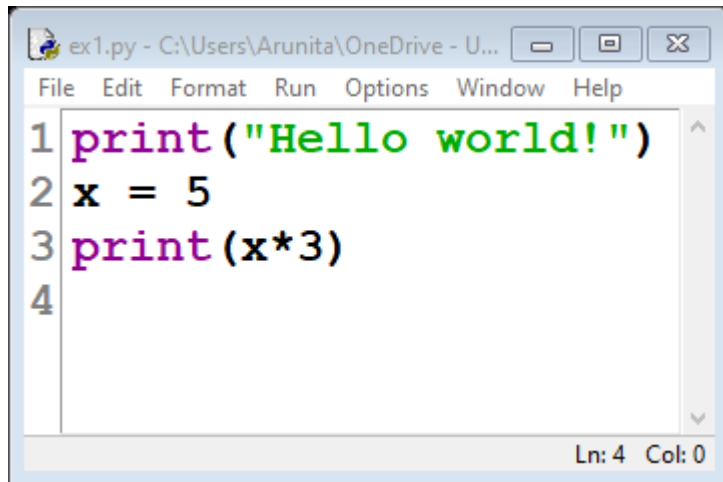
Editor

- Can be used to write your code and **save** it.
- Can run your saved code, using the '**Run**' menu
- Output is displayed in the **Shell** window. Note that output requires use of *print* or *write*



```
ex1.py - C:\Users\Arunita\OneDrive - U...
File Edit Format Run Options Window Help
1 print("Hello world!")
2 x = 5
3 print(x*3)
4
Ln: 4 Col: 0
```

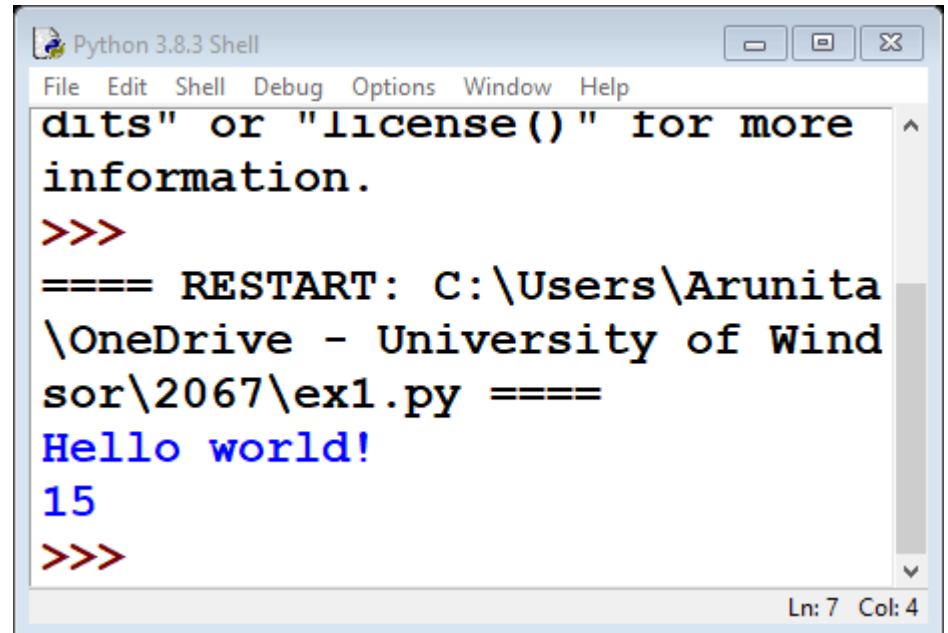
Create and Run a Simple Program



A screenshot of a Python IDE window titled "ex1.py - C:\Users\Arunita\OneDrive - U...". The window has a menu bar with "File", "Edit", "Format", "Run", "Options", "Window", and "Help". The code editor contains the following Python code:

```
1 print("Hello world!")
2 x = 5
3 print(x*3)
4
```

The status bar at the bottom right indicates "Ln: 4 Col: 0".



A screenshot of a "Python 3.8.3 Shell" window. The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The shell displays the following text:

```
...its" or "license()" for more
information.
>>>
==== RESTART: C:\Users\Arunita
\OneDrive - University of Wind
sor\2067\ex1.py ====
Hello world!
15
>>>
```

The status bar at the bottom right indicates "Ln: 7 Col: 4".

Task for self-practice

In this task, you will install Python, learn how to use IDLE, and create and run a simple program.

1. Install Python on your computer by downloading the latest version from the official website (<https://www.python.org/downloads/>).
2. Once Python is installed, open IDLE. IDLE is the Integrated Development Environment that comes with Python.
3. In IDLE, click on "File" and then "New File" to open a new text editor window.
4. Type the following code in the text editor window:

```
1 print("Hello, world!")
```



Task for self-practice (continued)

5. Save the file as "hello.py" in a location that is easy to remember (e.g., your desktop).
6. To run the program, click on "Run" in the IDLE menu bar and then click on "Run Module" or press F5.
 - You should see the following output in the Python Shell:

```
Hello, world!
```

- Now you need to write a program that greets the user with their name:
 1. Open IDLE and create a new file.
 2. Type the following code:

```
1 name = input("What is your name? ")
2 print("Hello, " + name + "!")
```



Task for self-practice (continued)

3. Save the file as greeting.py.
4. Run the program by pressing F5 or selecting Run > Run Module from the menu.
5. When prompted, enter your name and press Enter.

The program should greet you with a message like "Hello, John!" (assuming your name is John).

Please be advised that this particular task is not graded, and you are not required to submit it.



References

- [1] https://en.wikipedia.org/wiki/Computer_program
- [2] <https://www.britannica.com/technology/computer-program>
- [3] <https://dictionary.cambridge.org/dictionary/english/computer-program>
- [4] <https://whatis.techtarget.com/definition/instruction>

